

Chapter 3. Advanced Planning, Reasoning, and Scalable Execution in Agents

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at mcronin@oreilly.com.

In the last two chapters, you saw that AI agents aren't magic, they're engineered systems. But the techniques you'll learn in this chapter may start to feel close. As Arthur C. Clarke once said:

*Any sufficiently advanced technology is indistinguishable from magic.*¹

That sense of magic comes from what happens when agents stop reacting and instead start learning from their own experience or making smarter choices at test time. They are no longer non-player characters (NPCs), because they start to learn and adapt on their own when you apply the principles of reinforcement learning (RL) to LLMs. This is how Clarke's quote finds new meaning in the world of AI agents.

But what may look uncanny at first, actually comes from a set of clear mechanisms. This chapter explains those mechanisms in depth: you'll learn how RL builds a feedback loop between reasoning and outcomes, how tree-based search and adaptive planning lets agents simulate multiple futures before acting and decide if it's better to explore more solutions or rather to refine a potential solution.

Each of these mechanisms contributes to the illusion of self-improvement, which is of course, the refining process by which agents become better over time. Agents gather feedback, reflect on their choices, and refine their behavior in ways that look alive. Rewards, feedback loops, and dynamic replanning connect with RL frameworks such as [Agent](#)

[Reinforcement Training](#) (ART), Relative Universal LLM-Elicited Rewards ([RULER](#)) and [reinforcement learning for language models](#) (rLLM) to form a cycle of continuous refinement that spans across training and deployment.

In this chapter, you cross the threshold between design and autonomy. Your AI agents stop following scripts and start solving problems on their own. When foresight, reinforcement, and scalable execution work together this is the pivotal moment when you get the impression that your AI agents are no longer purely orchestrated. Here, you give your agents the awareness of choice and improvement via RL you build your system to learn, reason, and improve. In short, this chapter will help you design the emergence of agency for your agents.

Beyond Zero Sum Games: From Rewards to Reasoning

If you ever wondered where the learning in RL actually happens, in this section you'll learn where the story begins. Everything starts with *rewards*, the signals that tell an agent whether it did well or not. But rewards alone don't make an intelligent system. They simply create the feedback that drives it to improve.

At its core, RL is actually simple. Seriously, I mean it. Think about it for a moment, if you look closer, an agent takes an action, observes what happens, and uses that outcome to make a better choice next time. When this cycle repeats, patterns start to form. The agent begins to recognize what works and what doesn't. That is the moment when the loop between perception and adjustment closes, and behavior starts to look intentional. You already do this in real life. You use RL every day without thinking about it, either learning from your own experiences or how you teach your kids or pets.

The only difference is that for LLM agents this loop is in a different space; it isn't physical. It happens in language, thought, and reasoning. Every message, every tool call, and every piece of intermediate output becomes part of a trajectory. A *trajectory* is the full record of how an agent reached an answer, step by step, from the first prompt to the final result. [Figure 3-1](#) captures the fundamentals of this learning loop in a single glance: the moment where action turns into feedback and feedback turns into improvement.



Figure 3-1. Agent learning loop: from action to feedback to improvement

When you collect multiple trajectories, you get rollouts. Each *rollout* is one complete story of reasoning. Some reach the goal, some fail, but all of them teach the agent something about how its own decisions lead to different outcomes. RL turns these experiences into structure, by turning raw sequences of text into measurable cause and effect. [Table 3-1](#) gives you a conceptual compass of these concepts before we start looking into how you can apply them.

Table 3-1. RL Concepts

Concept	What it means	Why it matters for agents
Reward	A signal that tells the agent whether an outcome was good or bad.	Rewards close the loop between action and consequence. Without them, the agent has no sense of progress.
Trajectory	The full sequence of reasoning steps, actions, and tool calls that lead to an outcome.	A trajectory captures how the agent reached a decision, not just what it decided.
Rollout	A complete run of one or more trajectories for a task.	Rollouts record the agent's experiences so they can be compared, scored, and used for learning.
Policy	The model's internal decision rule that maps context to the next action.	RL updates the policy so the agent makes better choices over time.
Reward or Judge Model	A model that evaluates trajectories and assigns rewards based on quality or correctness.	Turns raw outcomes into learning signals that drive improvement.
Relative Evaluation	Comparing multiple rollouts to decide which performed best.	Removes the need for hand-crafted rewards and allows self-improvement from comparison.

The agent doesn't need a teacher to label every step. It only needs a way to compare its own trajectories and decide which ones lead to better reasoning. This is where reward models come in. Early systems used simple, hand-crafted rules, but that approach doesn't work for LLM-based agents. Modern agents use relative evaluation to learn directly from comparison. They generate several rollouts for the same task and rank them to see which reasoning path performed best.

Once the agent starts to do that, it moves beyond a zero sum game. It no longer tries to win a single round. It learns how to improve its entire process. The goal shifts from getting the answer right to reasoning more effectively.

This is the first real magic trick of RL for agents: the model begins to learn from its own trajectories. It starts to use rollouts as memory and feedback as guidance. What was once trial and error becomes deliberate improvement in your system.

Better in Groups: Why Relative Ranking Matters

A single trajectory tells you what happened, while a group of trajectories shows you what could have happened. Comparing them is where real learning begins.

In traditional RL, the model learns from numeric rewards that describe how well each attempt performed. But for language-based agents, those numbers are hard to define. You can't easily score creativity, coherence, or depth of thought with a single metric.

Relative ranking solves that problem. Instead of assigning absolute scores, you let the model compare several rollouts for the same task and decide which one worked best. The feedback doesn't have to be perfect. It only needs to be consistent within the group. If one trajectory is slightly better than another, that difference already carries enough information to guide improvement.

LEARNING ON A LEASH: KL DIVERGENCE

When you teach an agent to learn from its own experience, there is always a risk that it learns too fast or in the wrong direction. In RL, this problem is not philosophical, it's mathematical. If the new policy drifts too far from what the model already knows, it starts to forget useful behaviors.

The Kullback–Leibler divergence, or KL divergence, is the part of the equation that keeps that from happening. It measures how much the new probability distribution Q of the updated policy differs from the reference distribution P . In other words, it tells you how far the new “way of thinking” has moved from the old one.

If that distance becomes too large, the model begins to overfit to recent feedback and loses balance. By adding a KL penalty to the learning objective, you keep improvement under control. The agent can still explore and adapt, but not to the point of breaking its past understanding.

You can think of it as a leash on curiosity. The agent can wander, but it can't run away too far. Each update is a negotiation between two instincts: the drive to improve and the need to remain stable.

This idea forms the basis of Group Relative Policy Optimization (GRPO)² and Group Sequence Policy Optimization (GSPO)³. Both methods let agents learn from relative feedback rather than fixed targets. GRPO focuses on the token level, while GSPO refines the process at the sequence level, helping agents reason more efficiently step by step.

From Token-Based Rewards to Sequence-Based Learning

With GRPO and GSPO, learning becomes social in a sense. Each trajectory teaches the agent about the others. Success and failure stop being isolated events and turn into references. The agent no longer asks “Was I right?” but “Was this reasoning better than before?”

This shift changes everything. It turns RL from a scoring game into a process of reflection. The agent doesn’t just correct mistakes, it learns how to think and act better next time.

GRPO

GRPO builds on a simple idea: a model learns better when it can compare its own answers. Instead of scoring each response with an absolute reward, GRPO collects several responses to the same query, ranks them, and learns from those relative rankings. [Figure 3-2](#) shows a high-level overview of how this process works.

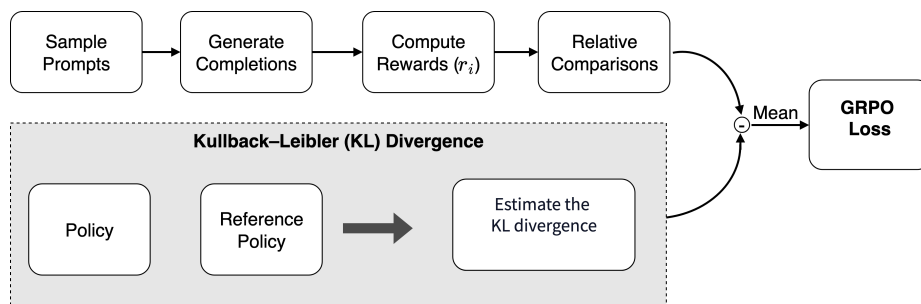


Figure 3-2. High-level GRPO mechanics: group-normalized rewards combined with a reference policy KL term.

The process begins by sampling prompts and generating multiple completions from the current policy. Each completion is then evaluated by a reward function or judge model, which assigns an individual score r_i . These scores are normalized within each group, centering and scaling their values to create a set of relative comparisons that serve as the update signal for the policy.

If you're wondering which common RL methods actually require an extra reward or judge model, [Table 3-2](#) makes this clear. PPO, as used in RLHF, combines a learned reward model with a critic for stability and variance reduction. DPO removes both, optimizing directly on pairwise preferences with an explicit KL to a reference. GRPO follows the PPO idea but replaces absolute rewards with relative rankings across groups, removing the critic while keeping ratio clipping and KL regularization. GSPO extends GRPO to sequence-level optimization, which improves stability on long or complex outputs.

Table 3-2. Which method is doing what?

Method	Reference policy	Reward or judge	Critic model	Optimization unit
PPO (RLHF)	Usually a frozen supervised model	Learned reward model R_ϕ from preferences	Yes, value head or separate critic	Token-level with advantages
DPO	Frozen reference π_{ref}	None learned at train time, uses pairwise human prefs (chosen vs rejected)	No critic	Sequence-level, pairwise preference loss
GRPO	Often $\pi_{\text{ref}} = \pi_{\text{old}}$ or a frozen ref	Judge or reward function, usually LLM-as-judge or programmatic scoring, per sample	No learned critic	Token-level, but advantages are group-normalized across multiple candidates
GSPO	Often π_{old} or a frozen ref	Judge or reward function, same spirit as GRPO	No critic	Sequence-level importance ratio, group-normalized

Each group of responses becomes a small ecosystem of experience. The model looks at its own work, compares outcomes, and updates its policy so that higher-ranked reasoning paths become more likely next time. This removes the need for a separate value model and stabilizes training through shared context.

Next, the KL divergence term is estimated between the current and reference policies. Before that, let's define how the model assigns probabilities

to each sequence. For a query q and a response o , the sequence likelihood is defined as:

$$\pi_{\theta}(o | q) = \prod_{t=1}^{|o|} \pi_{\theta}(o_t | q, o_{<t}),$$

where $|o|$ is the number of tokens in the response. This factorization reflects how an autoregressive model predicts each next token conditioned on all previous ones.

The KL divergence then measures how much the updated policy diverges from the reference one:

$$\mathbb{D}_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}}) = \mathbb{E}_{q, o \sim \pi_{\theta}(\cdot|q)} \left[\log \frac{\pi_{\theta}(o | q)}{\pi_{\text{ref}}(o | q)} \right].$$

In practice, this KL term acts as a regularizer that penalizes the model for drifting too far from its reference, while still allowing meaningful improvement. You can think of it as a leash that keeps the policy stable during training. Now, the group-based rewards and the KL penalty are combined into the GRPO objective. For each query q , the model samples a group of responses $\{o_i\}_{i=1}^G$ from the old policy $\pi_{\theta_{\text{old}}}$. Each response contributes a token-level importance ratio $w_{i,t}(\theta)$, which measures how much more (or less) likely the new policy is to generate the same token compared to the old one:

$$w_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}.$$

The reward for each response is normalized within the group, giving the relative advantage A_i :

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \dots, r_G)}{\text{std}(r_1, r_2, \dots, r_G)}.$$

With these elements, the GRPO objective becomes:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_{q, \{o_i\}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min(w_{i,t}(\theta) A_i, \text{clip}(w_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i) \right] - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta} \| \pi_{\text{ref}}).$$

This equation captures GRPO's core intuition mathematically, [Table 3-3](#) shows the conceptual overview of GRPO.

Table 3-3. GRPO Objective: what each part does

Term	What it means	Why it matters for agents
$w_{i,t}(\theta) A_i$	Encourages the model to increase the likelihood of higher-ranked responses.	Guides learning toward better reasoning paths within each group.
Clipping $\text{clip}(w_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon)$	Prevents large updates when the new policy drifts too far from the old one.	Stabilizes learning by limiting off-policy jumps that could cause collapse.
$-\beta \mathbb{D}_{\text{KL}}(\pi_\theta \pi_{\text{ref}})$	Penalizes the model for diverging too much from its reference policy.	Acts as a leash on exploration, keeping improvement balanced with stability.

This group-based normalization stabilizes learning by anchoring each update within its local context. As you learned in [“Learning on a Leash: KL Divergence”](#), the KL divergence puts a leash on the policy, so that improvement doesn’t lead to instability via too much divergence from the reference.

In practice, GRPO operates at the token level. The clipping and KL penalty are applied incrementally as the model generates each token, allowing the training signal to remain both adaptive and efficient.

GRPO’s strength lies in its simplicity: no critic model, no handcrafted reward scaling, only relative comparison within each group. This design made GRPO one of the first RL methods to scale language models reliably, before GSPO refined it further.

GSPO

GSPO takes the principles of GRPO further by refining how the model interprets and optimizes its experience. While GRPO evaluates reasoning at the token level, GSPO moves one level higher and optimizes entire sequences as units. [Figure 3-3](#) shows a high-level overview of the GSPO mechanics.

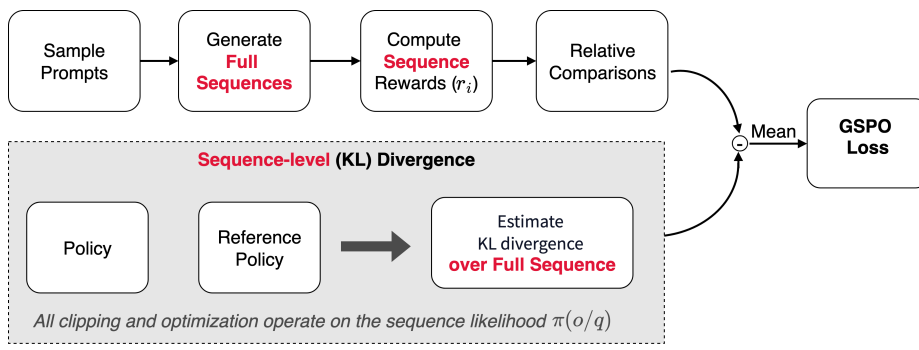


Figure 3-3. Unlike GRPO, which evaluates at the token level, GSPO computes rewards and KL divergence over complete sequences.

The motivation behind GSPO is straightforward: the reward belongs to the entire sequence, not to individual tokens. If the model receives feedback for the whole response, then the optimization should also act at that scale. By shifting the learning signal from tokens to sequences, GSPO aligns the unit of optimization with the unit of reward.

As before, the model starts from a query q and a set of responses $\{o_i\}_{i=1}^G$ sampled from the old policy $\pi_{\theta_{\text{old}}}$. But now, instead of token-level likelihood ratios, GSPO measures how much the new policy diverges from the old one across the entire response. This sequence-level importance ratio is defined as:

$$s_i(\theta) = \left(\frac{\pi_{\theta}(o_i | q)}{\pi_{\theta_{\text{old}}}(o_i | q)} \right)^{1/|o_i|} = \exp \left[\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} \right],$$

where $|o_i|$ denotes the number of tokens in the response. This length normalization keeps the ratio numerically stable, preventing longer outputs from producing disproportionately large updates.

The group-based advantage A_i remains the same as in GRPO and is computed as:

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \dots, r_G)}{\text{std}(r_1, r_2, \dots, r_G)}.$$

With these definitions, the GSPO objective becomes:

$$\mathcal{J}_{\text{GSPO}}(\theta) = \mathbb{E}_{q, \{o_i\}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) A_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) A_i \right) \right].$$

This objective follows the same logic as GRPO but applies the update at the sequence level instead of per token. By doing so, it eliminates the variance that can accumulate when token-level importance weights fluctuate independently within long responses. Implementations often include a sequence-level KL penalty analogously to GRPO, if omitted, early stopping and group normalization still stabilize training. [Table 3-4](#) illustrates the core components of GSPO.

Table 3-4. GSPO Objective: what each part does

Term	What it means	Why it matters for agents
$s_i(\theta) A_i$	Encourages the model to favor entire responses with higher relative rewards.	Aligns learning with full-sequence performance rather than token-by-token corrections.
Clipping $\text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon)$	Restrains overly large updates when the new policy differs too much from the old one.	Reduces instability and prevents gradient explosions on long outputs.
Length normalization o_i^{-1}	Scales updates according to response length.	Keeps learning consistent across short and long completions.

This shift from token-level to sequence-level optimization has a profound effect. It removes the instability that can emerge when token-level corrections introduce noise, especially in long responses or very large models. By clipping and rewarding entire responses instead of individual tokens, GSPO makes learning smoother, more reliable, and better aligned with how rewards are assigned in practice.

It also simplifies reinforcement learning infrastructure. Because GSPO operates on sequence likelihoods, it can often reuse probabilities already computed during inference rather than recomputing token-level likelihoods during training. This makes it both computationally efficient and stable across large-scale systems. [Table 3-5](#) compares both methods.

Table 3-5. GRPO vs. GSPO

Aspect	GRPO	GSPO
Learning signal	Learns from relative rankings of multiple responses to the same query.	Learns from sequence-level comparisons, weighting each full response by its overall likelihood.
Level of optimization	Token-level updates within each response.	Sequence-level updates across full responses.
Reward granularity	Same advantage shared across tokens, but importance ratios computed per token.	Advantage computed per sequence, importance ratios applied uniformly across all tokens.
Stability	Sensitive to noise accumulation over long outputs, may become unstable in very large or MoE models.	Significantly more stable, avoids token-level variance and prevents model collapse.
Clipping mechanism	Clips importance ratios per token to prevent large off-policy deviations.	Clips entire sequences, aligning reward and optimization scales.
Efficiency	Requires more frequent recomputation of token likelihoods during training.	Can reuse sequence-level likelihoods from inference, improving compute efficiency.
Infrastructure complexity	Needs value-model surrogates or routing replay for MoE stability.	Removes the need for extra stabilization strategies, simplifying large-scale RL training.
Practical effect	Enables reflection and learning from comparison but may struggle to scale.	Scales reliably to very large models and supports modern architectures like Qwen3 and MoE systems.

Together, GRPO and GSPO show how agents can learn not from explicit answers but from their own judgment. GRPO teaches the agent to compare outcomes and learn from relative success at the token level, while GSPO improves over GRPO by optimizing grouped policy sequences at the sequence level. This shifts evaluation from judging parts of an output to assessing the entire sequence under the policy. At this point, your agent has the essential components for self improvement. It can evaluate, compare, and refine. The next challenge is teaching it to apply these principles systematically, turning scattered experiences into structured trajectories that can be reused for continued growth.

Taking Off the Training Wheels: Teaching Agents How to Learn

By this point, your agent has seen a lot. Depending on your underlying language model, it may already have gathered experience, compared its own reasoning paths, and learned how to distinguish good decisions from poor ones. Yet general experience alone doesn't make a system reliable for your specific tasks. What truly matters is how you can teach your agent a new skill or become better at an already learned one. That is what this section is about.

If you think of your agent as a learner, using RL to teach it a new skill is the moment the training wheels come off. Every wobble, correction, and forward push becomes part of its evolving policy. Yes, your agent might wobble, crash, and correct, but that's how learning actually begins.

Through frameworks such as ART and RULER, your agent learns to refine itself through structured feedback loops. *ART* is an open-source framework designed to train agentic LLMs through structured rollouts centered on tool use and decision-making. It enhances an agent's performance and reliability by learning from experience, building on GRPO. *RULER* complements this approach as a general-purpose reward function that uses an LLM-as-judge to rank agent trajectories by quality, providing consistent feedback without handcrafted metrics.

Together, ART and RULER help you guide your agents toward what makes a good output purely from the task description, without any expected outputs required. The result is not just an agent that performs tasks, but one that develops a deeper understanding of how to improve itself both in context and when collaborating with other agents.

Beyond Isolation: Why Absolute Scoring is Easier

Reward design used to be one of the hardest parts of RL. Custom reward functions often broke down as tasks changed or scaled. LLM-as-judge systems were meant to fix that, but early attempts struggled with inconsistent scores and noisy calibration. RULER solves this by shifting from absolute scoring to relative comparison. In addition, as you already know, GRPO and GSPO normalize scores within each group, so only the relative rankings matter, not the absolute values.

RULER generates several trajectories for the same task. Each trajectory records the agent's reasoning, tool calls, and final outcome. Shared prefixes are removed, and only the unique parts are sent to an LLM-as-judge along with a simple rubric. The judge scores each trajectory between 0 and 1 based on goal completion and efficiency. Those scores are normalized within the group and used directly as rewards for a GRPO update. The judge can also explain its decisions, which helps you trace errors and

cluster low-performing runs. [Figure 3-4](#) shows how this process fits together.

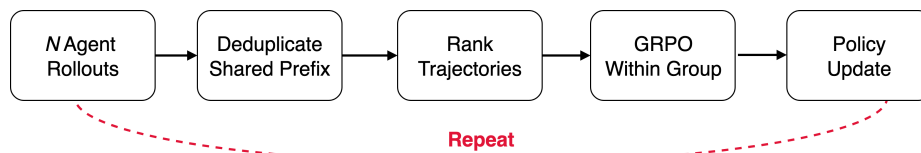


Figure 3-4. RULER training loop: trajectories are ranked by an LLM-as-judge, normalized by GRPO, and used to update the policy iteratively.

Let me give you a concrete coding example to understand how simple RULER actually is. Imagine you want to evaluate short poems about cats. You'll start by preparing the messages for the LLM, as in [Example 3-1](#).

Example 3-1. Shared message setup

```
initial_messages = [
    {"role": "system", "content": ""You are a poetic writer. Write short
                                     cat-themed poems that evoke emotion.""}
    {"role": "user", "content": ""Write a poem about cats observing the
                                   sky.""}
]
```

The next step is to create a few sample trajectories that differ in quality. [Example 3-2](#) defines a set of mocked trajectories that differ in quality, ranging from a well-written poem to an off-topic response. In a real RULER run, the agent would generate these outputs itself. Here, they're predefined so you can clearly see how the scoring process works.

Example 3-2. Create trajectories with different quality levels

```
good_trajectory = art.Trajectory(
    messages_and_choices=[
        *initial_messages,
        Choice(
            finish_reason="stop",
            index=0,
            message=ChatCompletionMessage(
                role="assistant",
                content=(
                    "Upon the roof, their eyes like stars,\n"
                    "They watch the sky from silver jars.\n"
                    "Each whisker twitches, soft delight,\n"
                    "As moons reflect their borrowed light."
                )
            )
        ]
    ),
    reward=0.0
)

mediocre_trajectory = art.Trajectory(
```

```

        messages_and_choices=[
            *initial_messages,
            Choice(
                finish_reason="stop",
                index=0,
                message=ChatCompletionMessage(
                    role="assistant",
                    content=(
                        "Cats sit and look at stars at night,\n"
                        "They think the stars are shiny and bright."
                    )
                )
            )
        ],
        reward=0.0
    )

    off_topic_trajectory = art.Trajectory(
        messages_and_choices=[
            *initial_messages,
            Choice(
                finish_reason="stop",
                index=0,
                message=ChatCompletionMessage(
                    role="assistant",
                    content=(
                        "Dogs are great companions under the moon. "
                        "They bark at stars and wag their tails happily."
                    )
                )
            )
        ],
        reward=0.0
    )

```

After the trajectories are generated, [Example 3-3](#) evaluates them in one pass. RULER scores and ranks the candidates so that the language model can act as a judge and select the strongest response.

Example 3-3. Score with RULER

```

group = art.TrajectoryGroup([good_trajectory, mediocre_trajectory,
                             off_topic_trajectory])
judged_group = await ruler_score_group(group, "openai/o3", debug=True)

```

With the group judged, [Example 3-4](#) simply retrieves the rewards and prints a sorted leaderboard. This makes it easy to see how each poem performed under RULER's evaluation.

Example 3-4. Show trajectory ranking

```

if judged_group:
    sorted_trajectories = sorted(judged_group.trajectories,

```

```

        key=lambda t: t.reward, reverse=True)
    for rank, traj in enumerate(sorted_trajectories, 1):
        messages = traj.messages()
        print(f"Rank {rank}: Score {traj.reward:.3f}")
        print(f"    Response: {messages[-1]['content'][:80]}...\n")

```

A typical output looks like this:

```

[RULER] Pretty-printed LLM choice JSON:
{
  'scores': [
    {
      'trajectory_id': '1',
      'explanation': 'Poetic, cat-focused, night-sky theme fulfilled with
                    vivid imagery; brief and compliant.',
      'score': 0.9
    },
    {
      'trajectory_id': '2',
      'explanation': 'Meets topic but very simplistic and minimally
                    evocative; partial fulfillment.',
      'score': 0.6
    },
    {
      'trajectory_id': '3',
      'explanation': 'Ignores cat theme, focuses on dogs; fails the
                    main instruction.',
      'score': 0.05
    }
  ]
}

```

Here is why this works:

- Comparing several candidates is easier for a language model than grading one in isolation. Differences become clear when options are viewed side by side.
- GRPO only requires rewards that are consistent within a group. It does not need global calibration, which keeps learning stable and scalable.

In practice, this setup removes the need for labeled data or hand built rules. It also accelerates convergence because partial improvements receive proportional credit. The result is a reward signal that is self-correcting, interpretable, and efficient.

From this example you can clearly see how RULER helps grade textual or other open-ended tasks such as explanations, reasoning chains, summaries, essays, customer interactions, and scientific analysis. Quality in these settings depends on clarity, completeness, depth of reasoning, tone, and creativity — none of which can be reduced to a single formula.

RULER uses a relative, model based comparison across multiple trajectory-

ries so the LLM-judge can rank subtle differences such as more coherent reasoning flow, fewer hallucinations, and clearer logic.

For deterministic problems such as the [countdown tasks](#), where the objective is to reach a specific target using a fixed set of numbers and arithmetic operations, the reward can be defined programmatically. A rule based reward works well here because correctness is binary and can be verified automatically.

However, when you extend the system to include explanatory or reflective reasoning, where the agent must describe how it arrived at a valid expression or justify each operation, the evaluation criteria become more nuanced. In these cases, a hybrid reward can be introduced. This mechanism combines the precision of a deterministic correctness check with the qualitative judgment of RULER, so that only valid mathematical solutions are scored while their reasoning quality, structure, and clarity still influence the final reward.

This concept is not limited to numerical reasoning. The same hybrid reward can be applied to code agents, where functional correctness can be checked programmatically but style, readability, or documentation quality may require language model evaluation. It can also be extended to text generation tasks by mapping objective measures, such as factual accuracy or constraint satisfaction, to scalar correctness scores and combining them with RULER style assessments.

In essence, the hybrid reward offers an easy way to merge symbolic verification with language based evaluation, creating reward functions that balance precision and interpretability. The next section demonstrates how this can be implemented in practice using ART and GRPO to train an agent to solve the countdown tasks.

The ART of Learning from Experience

Now that you know how RULER works, you're ready to take your agent to the next step and actually produce trajectories for a task using ART to help it improve. In the following example, you'll teach your agent to solve the mathematical countdown tasks with a hybrid reward. The setup combines deterministic rewards for numerical correctness with RULER-based scoring for reasoning quality, allowing your agent to learn both precision and clarity.

NOTE

Note that I focus on the key code snippets here and omit helper functions or minor sections to highlight the core logic. The full runnable code is available in the [book's repository](#).

You begin by preparing the dataset (omitted) and a trainable model ([Example 3-5](#)) with a local backend ([Example 3-6](#)).

Example 3-5. Set up model and model configurations

```
model = art.TrainableModel(  
    name="countdown-agent-001",  
    project="countdown-agent",  
    base_model="Qwen/Qwen2.5-7B-Instruct",  
)
```

Example 3-6. Set up and register backend

```
backend = LocalBackend(in_process=True, path="./.art")  
await model.register(backend)
```

Next you need to define the task interface: simple tools to read the current context, search for a valid expression, and return the final answer so ART can score the trajectory. I omitted this part here too, but in the notebook you'll find this under the section: exact search tool with fractions.

Before you can run rollouts, you should add a programmatic judge for binary correctness. This gives you the hard reward that enforces exact solutions. [Example 3-7](#) is an example of such a reward.

Example 3-7. Add deterministic judge for correctness

```
def judge_countdown_expression(  
    target: int,  
    allowed_nums: List[int],  
    expr: str,  
    enforce_integer_intermediates: bool = False,  
) -> Tuple[float, Optional[float], Optional[str]]:  
  
    expr = expr.strip()  
    if not expr:  
        return 0.0, None, "empty expression"  
    if re.search(r"^[^0-9\+\-\*\\/\(\)\.\s]", expr): ❶  
        return 0.0, None, "invalid characters"  
  
    used = numbers_used_in_expr(expr) ❷  
  
    def counts(xs):  
        d = {}  
        for v in xs:  
            d[v] = d.get(v, 0) + 1  
        return d  
  
    allowed_counts = counts([int(x) for x in allowed_nums])  
    used_counts = counts(used)  
  
    for v, c in used_counts.items():
```

```

        if v not in allowed_counts or c > allowed_counts[v]:
            return 0.0, None, f"illegal number usage: {v}"
    try:
        if enforce_integer_intermediates:
            val_frac = _eval_ast_fraction(ast.parse(expr, mode="eval"))
            if val_frac == Fraction(target, 1):
                return 1.0, float(val_frac), None
            return 0.0, float(val_frac), "wrong value"
        else:
            val = safe_eval_expr(expr)
            if abs(val - target) < 1e-9:
                return 1.0, val, None
            return 0.0, val, "wrong value"
    except ZeroDivisionError:
        return 0.0, None, "division by zero"
    except Exception as e:
        return 0.0, None, f"eval error: {e}"

```

❶ Validate characters to be safe.

❷ Check number usage.

In [Example 3-8](#) you create scenarios from the dataset so each rollout has a clear prompt and identifiers.

Example 3-8. Build scenarios

```

class Scenario(BaseModel):
    id: str
    target: int
    nums: List[int]
    question: str

def mk_scenario(row, idx: int) -> Scenario:
    tgt = int(row["target"])
    nums = [int(x) for x in row["nums"]]
    q = """Using only the numbers each at most once, write a valid
        arithmetic expression with +, -, *, or / that evaluates
        exactly to the target. Return only the expression string."""

    return Scenario(id=f"cd_{idx}", target=tgt, nums=nums, question=q)

train_scenarios = [mk_scenario(train_ds[i], i) for i in range(len(train_ds))]
test_scenarios = [mk_scenario(test_ds[i], i) for i in range(len(test_ds))]

```

With data, tools, and judge in place, you can define your rollout.

[Example 3-9](#) runs one task end to end, allows tool use, records the final answer, and writes rewards and metrics on the trajectory for ART.

Example 3-9. Create rollout to capture trajectories

```

class ProjectTrajectory(art.Trajectory):
    final_answer: Optional[FinalAnswer] = None ❶

```

```

class CountdownScenario(BaseModel):
    step: int
    scenario: Scenario

@weave.op
async def rollout(model: art.Model, cd: CountdownScenario) -> ProjectTrajectory:
    scn = cd.scenario

    _nonlocal_final["value"] = None ❷
    _current_scenario["target"] = scn.target
    _current_scenario["nums"] = scn.nums
    _current_scenario["id"] = scn.id

    nums_str = ",".join(map(str, scn.nums))
    traj = ProjectTrajectory(
        reward=0.0,
        messages_and_choices=[],
        metadata={"scenario_id": scn.id,
                  "target": float(scn.target), "nums": nums_str},
    )
    if traj.metrics is None:
        traj.metrics = {}

    system_prompt = dedent(f"""
        You are a math agent for Countdown tasks.

        Goal: produce one expression that evaluates exactly to {scn.target}
        using only the numbers {scn.nums} each at most once.
        Operators: +, -, *, /. Parentheses are allowed.

        Tools:
        - read_countdown_context()
        - countdown_search_tool(nums=[...], target=...,
                                require_integer_intermediates=True)
        - return_final_answer_tool(answer=EXPR, reference_ids=[...])

        Process:
        1) Call read_countdown_context to confirm inputs.
        2) Call countdown_search_tool. If it returns a non empty expression,
          pass it verbatim to return_final_answer_tool.
        3) If it returns empty, reason and still obey number usage.
        The final tool must receive ONLY the expression string.
        """)

    tools = [read_countdown_context, countdown_search_tool, return_final_answer_tool]
    chat = init_chat_model(model.name, temperature=0.6, top_p=0.9, max_tokens=1024)
    agent = create_react_agent(chat, tools) ❸

    try:
        config = {"configurable": {"thread_id": str(uuid.uuid4()), ❹
                                   "seed": random.randint(0, 1_000_000)},
                  "recursion_limit": MAX_TURNS} ❺

        await agent.ainvoke(
            {"messages": [SystemMessage(content=system_prompt),
                          HumanMessage(content=scn.question)]},

```

```

        config=config,
    )

    if _nonlocal_final["value"]:
        traj.final_answer = _nonlocal_final["value"] ❹
        reward, val, reason = judge_countdown_expression(
            scn.target, scn.nums, traj.final_answer.answer,
            enforce_integer_intermediates=True
        )
        traj.reward = float(reward) ❺
        traj.metrics["correct"] = float(reward)
        traj.metrics["value"] = float(val) if isinstance(val,
            (int, float)) else float("nan")
        traj.metadata["last_reason"] = str(reason) if reason else ""

        traj.metadata["rubric"] = ( ❸
            "Score higher if the expression (1) hits the target, "
            "(2) uses each provided number at most once, "
            "(3) uses fewer operations, "
            "(4) avoids redundant parentheses and trivial +/-0 or *1 tricks, "
            "(5) avoids division when +, -, * suffice."
        )

    return traj

```

- ❶ Extend the trajectory with a typed slot for the agent's final tool output.
- ❷ Shared scratchpad objects that your tools read/write.
- ❸ ReAct-style agent wired with your 3 tools.
- ❹ Unique thread/seed per rollout → reproducibility and isolation.
- ❺ Safety against infinite tool loops.
- ❻ Persist the tool-returned answer on the trajectory.
- ❼ Hard reward from deterministic judge.
- ❽ Rubric used by RULER's LLM-as-judge.

[Example 3-10](#) shows you how to apply RULER within each group.

Example 3-10. Use RULER to score each group

```

ruler_model_id = "openai/gpt-4.1"
ruler_groups = []
for group in judged:
    try:
        rg = await ruler_score_group(group, ruler_model_id, debug=True) ❶
    except Exception:
        rg = None
    ruler_groups.append(rg if rg is not None else group) ❷

```

❶ Ask the judge to score/rank within each group.

❷ Fall back to the unjudged group if judging fails.

[Example 3-11](#) combines both signals to shape learning beyond correctness. Only correct solutions receive credit, then RULER ranks them to refine the reward.

Example 3-11. Combining correctness with RULER using a hybrid reward

```
alpha, beta = 0.7, 0.3 ❶

def _combine(groups, alpha: float = 1.0, beta: float = 1.0):
    for g in groups:
        raw = []
        for t in g.trajectories:
            try:
                raw.append(float(t.reward)) ❷
            except Exception:
                raw.append(0.0)

        rmin = min(raw) if raw else 0.0
        rmax = max(raw) if raw else 0.0
        span = (rmax - rmin) if (rmax > rmin) else 1.0 ❸

        for t in g.trajectories:
            if t.metrics is None:
                t.metrics = {}
            if t.metadata is None:
                t.metadata = {}

            gate = float(t.metrics.get("correct", 0.0) or 0.0) ❹
            gate = 1.0 if gate >= 0.5 else 0.0

            try:
                ruler_raw = float(t.reward)
            except Exception:
                ruler_raw = 0.0
            ruler_norm = (ruler_raw - rmin) / span
            ruler_norm = max(0.0, min(1.0, ruler_norm)) if ruler_norm == ruler_norm else 0.0 ❺

            final_reward = gate * (alpha + beta * ruler_norm) ❻
            final_reward = max(0.0, min(1.0, final_reward))

            t.metadata["ruler_score_raw"] = float(ruler_raw) ❼
            t.metrics["ruler_norm"] = float(ruler_norm)
            t.metrics["final_reward"] = float(final_reward)
            t.reward = float(final_reward) ❽

    return groups

hybrid_groups = _combine(ruler_groups, alpha=alpha, beta=beta)
```

- ❶ Weight of correctness vs. quality; adjust for your own task.
- ❷ Use judged rewards as the quality signal.
- ❸ Robust group-wise normalization (avoids div-by-zero).
- ❹ Binary gate from deterministic judge ensures precision first.
- ❺ Clip and NaN-guard.
- ❻ Simple, interpretable shaping formula.
- ❼ Keep both raw and normalized scores.
- ❽ Overwrite trajectory reward with the hybrid reward used for training.

[Example 3-12](#) sets up and runs the training loop. ART iterates over scenarios, collects trajectory groups, applies the hybrid reward, and updates the policy.

Example 3-12. Run training loop

```
training_config = {
    "groups_per_step": 4, ❶
    "num_epochs": 1,
    "rollouts_per_group": 2, ❷
    "learning_rate": 1e-5,
    "max_steps": 3, ❸
}

train_slice = train_scenarios[:64] ❹

training_iterator = iterate_dataset(
    train_slice,
    groups_per_step=training_config["groups_per_step"],
    num_epochs=training_config["num_epochs"],
    initial_step=await model.get_step(), ❺
)

for batch in training_iterator:
    print(f"Training step {batch.step}, epoch {batch.epoch}")
    groups = []
    for scn in batch.items:
        group = art.TrajectoryGroup(
            [wrap_rollout(model, rollout)(model,
            CountdownScenario(step=batch.step, scenario=scn))
            for _ in range(training_config["rollouts_per_group"])] )
        groups.append(group)

    finished = await art.gather_trajectory_groups( ❻
        groups, pbar_desc="gather",
        max_exceptions=training_config["rollouts_per_group"] * len(batch.items)
```

```

    )

    await model.train( ❷
        hybrid_groups,
        config=art.TrainConfig(learning_rate=training_config["learning_rate"],
                               _config={"logprob_calculation_chunk_size": 8},
        )
    )

```

- ❶ How many scenarios per optimizer step.
- ❷ How many rollouts per scenario (the comparison set).
- ❸ Keep small while debugging to avoid runaway training.
- ❹ Small slice for a fast first pass.
- ❺ Resume-safe: read current global step.
- ❻ Async gather lets slow rollouts finish without blocking the loop.
- ❼ Apply GRPO update using the hybrid rewards.

To test the trained agent, you can run [Example 3-13](#) on a few held out items.

Example 3-13. Test the trained agent

```

for scn in test_scenarios[:10]:
    res = await wrap_rollout(model, rollout)(model,
        CountdownScenario(step=0, scenario=scn)) ❶
    expr = res.final_answer.answer if res.final_answer else None ❷
    reward, val, reason = judge_countdown_expression(scn.target, scn.nums,
        expr or "", enforce_integer_intermediates=True) ❸
    status = "CORRECT" if reward == 1.0 else f"WRONG ({reason})"
    print(f"nums={scn.nums} target={scn.target} -> {expr} => {status}")

```

- ❶ Reuse the trained policy + identical tool loop for evaluation.
- ❷ Pull the final tool answer if present.
- ❸ Same hard judge as in training ensures apples-to-apples evaluation.

This results in the following print-out:

```

nums=[22, 89, 16] target=51 -> None => WRONG (empty expression)
nums=[85, 45, 75, 10] target=25 -> ((75-10)-(85-45)) => CORRECT
nums=[68, 96, 50, 3] target=75 -> ((50-3)-(68-96)) => CORRECT
nums=[49, 94, 73, 40] target=12 -> ((40-73)-(49-94)) => CORRECT
nums=[40, 79, 73] target=34 -> (73+(40-79)) => CORRECT
nums=[21, 1, 33, 2] target=40 -> (33+(21/(1+2))) => CORRECT
nums=[29, 1, 4, 46] target=66 -> ((4*(29-1))-46) => CORRECT
nums=[19, 97, 98] target=18 -> ((19+97)-98) => CORRECT

```



```
nums=[55, 76, 1] target=22 -> (1-(55-76)) => CORRECT  
nums=[31, 4, 16, 36] target=20 -> None => WRONG (empty expression)
```

This clearly shows that the agent successfully learned how to solve the countdown tasks, even though I just used a small split of the dataset.

Test Time Compute: Balancing Size with Thought

There is a limit to how far you can go by scaling model size alone. Training ever-larger language models demands enormous resources and quickly becomes unsustainable. Yet, reasoning ability doesn't come purely from size. Recent research shows that even smaller models can perform exceptionally well if they are given more time to think at test time.

Instead of adding more parameters, you can add more computation during inference. You can let the model reason longer, explore multiple ideas, and refine its answers before committing to a final one. This approach changes how you think about scale: not as more neurons, but as more deliberate reasoning. In short, so far you've taught your agent to learn, but now you teach it to think. This is where your learning systems turn into thinking systems.

This can mean sampling multiple reasoning paths, reflecting on intermediate steps, or revising answers based on feedback. The result is an agent that learns to spend compute where it matters. It can simulate alternative outcomes, evaluate them, and refine its reasoning dynamically. This idea of scaling thinking time rather than model size is one of the most promising paths toward more capable and efficient agents.

To support this shift, preference and reward models take on a new role. They no longer serve only during training but become part of the inference process itself. Instead of scoring a single final answer, they guide reasoning as it unfolds. Each step or branch in the reasoning process can be evaluated and compared, much like a human reviewing a line of argument or debugging code. This feedback loop allows smaller models to match or even outperform larger ones on complex reasoning tasks, especially when resources are limited or when models must run efficiently on local devices.

To scale test-time compute, you can use tree search algorithms which helps your agents allocate reasoning effort adaptively. Among these, Monte Carlo Tree Search (MCTS) is one of the most powerful. *MCTS* allows an LLM to explore a structured space of possible answers and gradually converge on the best one. It balances curiosity and focus, searching just widely enough to discover new ideas and deeply enough to refine them.

From Sampling to Search: Balancing Exploration and Exploitation

Before looking more closely at adaptive algorithms, it helps to start with two simpler strategies. There are two methods that set the perfect stage for you to understand the tradeoff between exploring more answers or going deeper into a specific answer: repeated sampling and sequential sampling, respectively. In *repeated sampling*, shown in [Figure 3-5](#), the model draws multiple independent answers for the same prompt. This approach only explores breadth, producing variety but no refinement. In contrast, *sequential refinement* focuses on depth. It starts from one initial answer and repeatedly improves it. Both methods are useful, but each only covers half of what real reasoning requires.

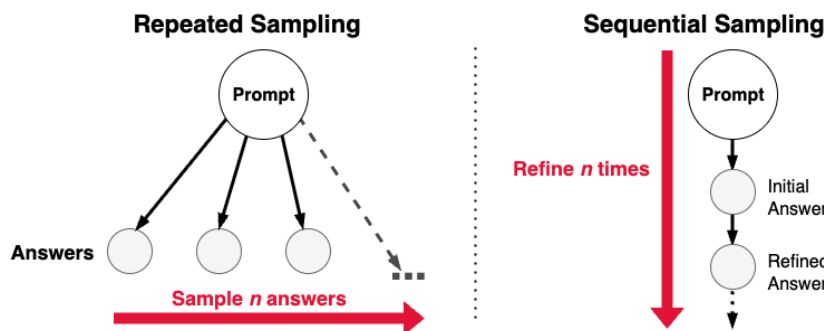


Figure 3-5. Flat sampling strategies: repeated sampling versus sequential refinement

MCTS builds on this concept by building a search tree through four recurring steps. It begins with selection, where the algorithm traverses the tree from the root, choosing branches that balance exploration and exploitation according to a scoring rule such as the Upper Confidence Bound for Trees (UCT). This balance, known in RL as the *exploration–exploitation tradeoff*, represents the tension between gathering new information to discover potentially better solutions (exploration) and using the information already collected to maximize performance (exploitation). With UCT the value for expansion is selected by the next iteration. The UCT of a child state s is calculated as follows:

$$\text{UCT}(s) = V(s) + c\sqrt{\frac{\ln N(p)}{N(s)}}$$

Here, $V(s)$ is the estimate of your node s , $N(s)$ is the visit count of your node s , $N(p)$ counts the visits of your parent node, and c stands for the exploration weight you can set. [Example 3-14](#) shows how to implement this in Python.

Example 3-14. UCT implementation

```
def upper_confidence_bound(self, exploration_weight=1.0):
    if self.parent is None:
        raise ValueError("Cannot obtain UCT from root node")
    if self.visits == 0:
        return float("inf")
```

```

parent_visits = max(1, self.parent.visits)
average_reward = self.value / self.visits
exploration_term = math.sqrt(math.log(parent_visits) / self.visits)
return average_reward + exploration_weight * exploration_term

```

Once a promising branch is found, it moves to expansion, where new child nodes are created to represent new candidate solutions. From there, the model performs a simulation, also called a rollout, where it generates and evaluates a full solution from that node. Finally, backpropagation updates all parent nodes with the outcome, allowing the system to refine its understanding of which directions are most promising. [Figure 3-6](#) illustrates this loop of planning, evaluation, and reflection.

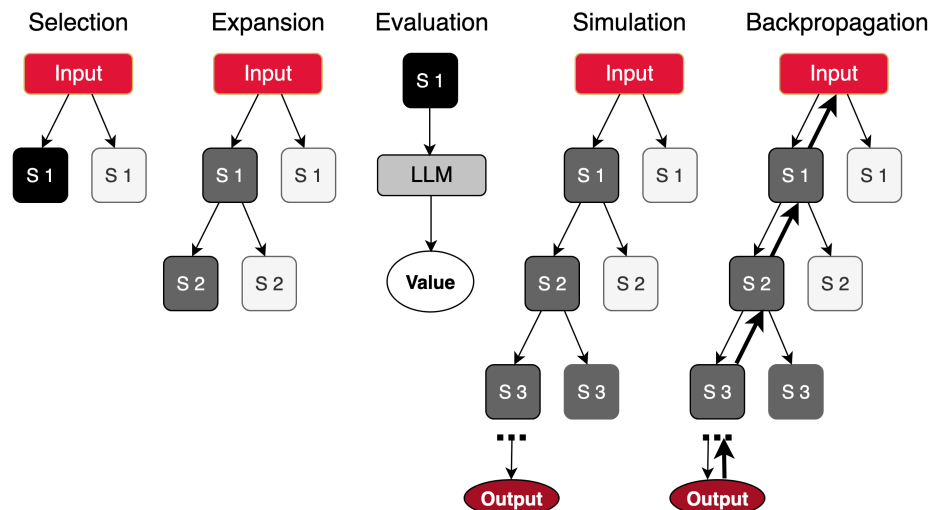


Figure 3-6. Monte Carlo Tree Search applied to LLM reasoning

At test time, the model's reasoning budget is spent running this loop: selecting, expanding, simulating, and backpropagating. The LLM generates candidate answers, evaluates them, and uses those evaluations to guide the next move. This process lets the model explore more possibilities than simple one-pass decoding, producing solutions that are both higher in quality and more explainable.

Adaptive Branching Monte Carlo Tree Search (AB-MCTS)

Adaptive Branching Monte Carlo Tree Search (AB-MCTS)⁴ extends standard MCTS by dynamically deciding whether to explore new branches or to refine existing ones. This adaptive decision process represents the classic exploration–exploitation trade-off in RL.

This flexibility is essential for your agents, because even a single prompt can generate an infinite variety of responses. A fixed branching factor, as used in standard MCTS, limits this potential. AB-MCTS overcomes that by allowing the tree to grow as needed. [Figure 3-7](#) shows how it differs from traditional MCTS.

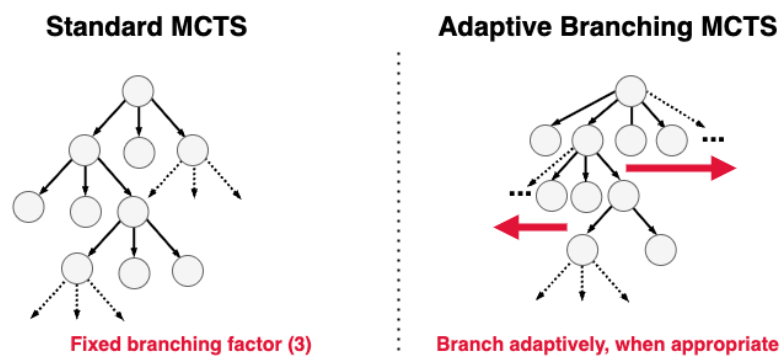


Figure 3-7. Comparison of MCTS with AB-MCTS

AB-MCTS introduces a *GEN node* under every tree node, when selected, this node signals that the model should generate a new candidate response, effectively creating a new branch. Whether to generate a new path or refine an existing one is guided by Thompson sampling, a Bayesian strategy that balances exploration and exploitation based on uncertainty.

There are two main variants. AB-MCTS-M (mixed model) uses a node-specific Bayesian model to estimate scores for both GEN and existing nodes. It draws on observed scores from subtrees to build posterior predictive distributions that guide search decisions. This allows the model to infer likely rewards even for paths that have not yet been explored, enabling principled decision-making with limited data. [Figure 3-8](#) illustrates this mechanism.

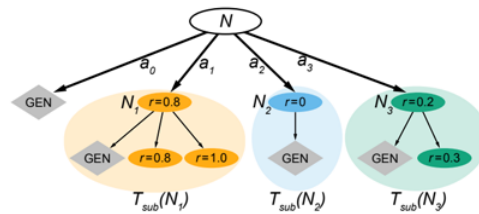


Figure 3-8. Tree structure and posterior predictive distributions for AB-MCTS-M

The second variant, AB-MCTS-A (node aggregation), is simpler and closer to standard MCTS. It introduces a *CONT node* under each answer node to represent refinement. Each node can either generate new candidates (via GEN) or continue improving an existing one (via CONT). This setup limits the computational cost while keeping the search flexible. Depending on how rewards are distributed, AB-MCTS-A can use Gaussian priors or Beta priors to normalize scores between 0 and 1. [Figure 3-9](#) shows how such a tree can evolve.

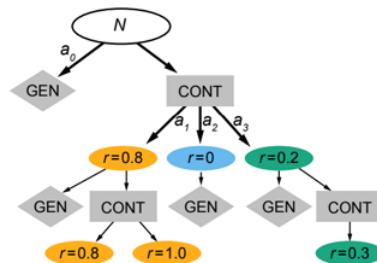


Figure 3-9. AB-MCTS-A tree structure with CONT and GEN nodes

Both AB-MCTS variants rely on Bayesian updating to guide the search. The LLM generates, evaluates, and refines responses iteratively, distributing its computational effort where it brings the greatest gain. This allows the model to reason adaptively: exploring when uncertain, exploiting when confident.

To illustrate, the next example uses the open source [TreeQuest library](#) to refine Python code for the Fibonacci sequence. It shows how tree search balances breadth and depth to improve an LLM agent's answer step by step.

[Example 3-15](#) creates the first candidate and scores it. It drafts a solution from scratch, calls the judge, and returns a `State` that MCTS can treat as a root child.

Example 3-15. Generate initial answer

```
def initial_generation() -> State:
    prompt = "Q: Write code in Python for the Fibonacci sequence. \nA:"
    response = client.chat.completions.create(
        model="gpt-4o", ❶
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7, ❷
    )
    answer = response.choices[0].message.content.strip()
    score = evaluate_answer(answer) ❸
    return State(llm_answer=answer, score=score)
```

- ❶ Use a model that supports JSON style response formatting for judging later.
- ❷ A modest temperature supports variation for future branches.
- ❸ Self-evaluates the answer to seed the root children with scores.

Next, [Example 3-16](#) performs a targeted improvement pass on an existing answer and re-scores it.

Example 3-16. Refine answer

```
def refine_answer(llm_answer: str, score: float) -> State:
    prompt = f"""The current answer is:\n\n{llm_answer}\n\nPlease improve this
        answer to be more informative, accurate, and clear."""
    response = client.chat.completions.create(
        model="gpt-4o", ❶
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
    )
    refined = response.choices[0].message.content.strip()
    score = evaluate_answer(refined) ❷
    return State(llm_answer=refined, score=score)
```

❶ Again, use a model that supports JSON style response

❷ Always re-score the refined answer to update the search tree.

[Example 3-17](#) asks an LLM judge to return a JSON object with a single `score` key. Uses the OpenAI client with structured parsing into `ScoreResponse`. On any error it falls back to a neutral score to keep the search momentum.

Example 3-17. LLM judge with structured parsing

```
def evaluate_answer(answer: str) -> float:
    prompt = (
        f"Evaluate the quality of this answer on a scale from 0 to 1.\n"
        f"Return a JSON object like {\"score\": 0.92}}.\n\n"
        f"Answer:\n{answer}"
    )

    try:
        completion = client.chat.completions.parse(
            model="gpt-4o",
            messages=[{"role": "user", "content": prompt}],
            response_format=ScoreResponse, ❶
        )
        return completion.choices[0].message.parsed.score
    except Exception as e:
        print(f"[Evaluation error] {e}")
        return 0.5 ❷
```

❶ Structured parse into `ScoreResponse` prevents brittle string parsing.

❷ Neutral fallback keeps the search progressing when judging fails.

If there is no `parent_state`, [Example 3-18](#) creates the initial candidate, or otherwise, refines the parent answer.

Example 3-18. Generation entry point for TreeQuest

```
def generate(parent_state: State | None) -> tuple[State, float]:
    if parent_state is None:
        return initial_generation(), initial_generation().score ❶
    return refine_answer(parent_state.llm_answer,
                        parent_state.score), parent_state.score ❷
```

❶ First call produces the initial root child that AB-MCTS can expand.

❷ Refinement corresponds to a CONT choice under AB-MCTS-A or a selected existing node under AB-MCTS-M. The second return value can be used by the library to compute deltas or edge utilities.

[Example 3-19](#) implements and runs the AB-MCTS loop.

Example 3-19. TreeQuest loop

```
algo = tq.ABMCTSA()
search_tree = algo.init_tree()

for i in range(5): ❶
    search_tree = algo.step(search_tree, {'LLM-Refine': generate})
    if (i + 1) % 5 == 0: ❷
        best, _ = tq.top_k(search_tree, algo, k=1)[0]
        print(f"[Step {i+1}] Best so far: {best.llm_answer} (score={best.score})")

best_state, _ = tq.top_k(search_tree, algo, k=1)[0] ❸
print(f"\n Final Best Answer: {best_state.llm_answer} (score={best_state.score})")
```

❶ Run a small number of steps (five here). Increase gradually in real use.

❷ Inspect best-so-far periodically.

❸ Final selection.

Part of the print-out looks as follows for me:

```
[Step 5] Best so far: Certainly! Let's refine the explanation and the code
to make it more informative, accurate, and clear.

### Improved Code
Here's the revised version of the code with additional comments for clarity:

```python
def fibonacci(n):
 """
 Generate the first n terms of the Fibonacci sequence.

 Parameters:
 n (int): The number of terms to generate.

 Returns:
 list: A list containing the first n terms of the Fibonacci sequence.
 """
 # Initialize the sequence list and the first two Fibonacci numbers
 sequence = []
 a, b = 0, 1

 # Generate Fibonacci numbers up to n terms
 while len(sequence) < n:
 sequence.append(a) # Append the current number to the sequence
 a, b = b, a + b # Update a + b to the next two numbers in the sequence
 return sequence
```

As you can see, I chose five improvement steps, and this is exactly what you see reflected here. Tree search shows what it means to scale reasoning instead of model size. By letting your agents explore, evaluate, and refine, you trade raw capacity for structured and guided thought. With algorithms like AB-MCTS, test-time compute reasoning becomes an active process, not a single pass.

## Enabling General-Purpose Agentic Programs Through Post-Training and Search-Based Agents

As you’ve seen, RL gives your agents the ability to improve beyond static prompting, closing the loop between reasoning, action, and feedback. Frameworks like ART and RULER make this process practical: they let your agents learn from structured trajectories and relative feedback instead of fixed labels or handcrafted rewards. But as your agents grow more complex, combining multiple reasoning roles, branching workflows, or search-based strategies, training these systems becomes your next challenge. This is where rLLM enters the picture as a valuable tool in your agent-improvement toolbox. In this section, training becomes orchestration. You teach multiple LLM-based agents how to divide work, judge outcomes, perform conditional branching and parallel execution, and coordinate tree search as a team so they can improve together.

rLLM is an open-source framework for post-training language agents through RL. It allows you to design custom agents and environments, train them with RL algorithms such as GRPO, and deploy them for production-scale workloads. Conceptually, it builds upon the same principles you already learned with RULER and ART, but extends them to a higher level of abstraction. Instead of training a single policy, rLLM enables entire agentic workflows to learn collectively.

At its core, rLLM introduces a workflow engine and a trainer that together generalize the RL process. The workflow engine executes complex reasoning or multi-agent programs such as solver–judge systems and planner–executor loops. It also enables you to use tree-search-based agents with MCTS, while collecting trajectories from each component. The trainer component aggregates these trajectories, computes relative advantages, and applies updates with KL regularization to ensure stability in your system. rLLM helps you to transform your individual agents into a trainable and collaborative system. [Table 3-6](#) compares each of the three frameworks, to make their distinct yet complementary role within the RL stack for agentic systems clearer.



Table 3-6. Frameworks for RL-based agent training

Framework	Core focus	Learning mechanism	Scope
RULER	Relative evaluation via LLM-as-judge	Converts qualitative reasoning into normalized quantitative feedback	Reward modeling and judgment
ART	Reinforcement learning for tool-using agents	Structured rollouts and GRPO-based policy updates	Behavioral optimization and reliability
rLLM	Reinforcement learning for agentic workflows	Unified training across reasoning, judgment, and search processes	System-level learning and coordination

rLLM integrates and extends the mechanisms you’ve already seen in GRPO. It also works seamlessly with methods like MCTS. Each node in a search tree becomes a reasoning step that can be optimized through reinforcement learning. The phases of selection, expansion, simulation, and backpropagation ([Figure 3-6](#)) all generate trajectories with measurable outcomes. Within rLLM, these trajectories can be grouped, ranked, and refined so your agents gradually learn how to search more effectively instead of relying on fixed heuristics.

rLLM helps you move from theoretical reinforcement learning setups to practical reasoning systems. The same control flow you use for inference, whether it’s a planner and executor working together, a solver and judge comparing results, or a search tree exploring different solutions, can now serve as your training loop. This bridges the gap between design and deployment, because the learning process naturally adapts to the way your agents already reason.

rLLM allows you to train any agentic system, no matter how complex it is. You simply define a workflow by inheriting from the `WorkflowBase` class, implementing your logic within a `run()` method, and returning an `Episode` object that captures all trajectories and rewards. The framework then orchestrates execution, retry logic, trajectory collection, and PPO/GRPO optimization, so you can focus entirely on your agent’s reasoning structure rather than on the surrounding infrastructure to train it.

To illustrate this idea, the [book’s repository](#) includes a notebook demonstrating a solver–judge workflow, a setup where solver agent propose answers and a judge agent evaluates them to select the best one. Each solver’s reward depends on its solution’s correctness, while the judge’s reward reflects whether it chose the correct solution. By bundling all trajectories into a single `Episode`, both solver and judge can be trained jointly,

allowing the system to evolve as one coordinated reasoning process, this process can also be applied to improve the coordination between MAS.

In the example notebook, this structure is applied to the 24-game, a classic mathematical reasoning challenge. The task provides four numbers, for example, 3, 3, 4, and 5, and asks the agent to combine them using arithmetic operations (+, -, ×, ÷) to reach exactly 24. The solver agents attempt different expressions, and the judge evaluates them. Over time, RL helps both components refine their reasoning strategies, illustrating how rLLM's workflow design enables end-to-end training across reasoning, judgment, and evaluation.

This way of training marks a clear step forward. Rather than fine-tuning models in isolation and combining them afterward, you can teach the entire reasoning architecture to grow as a single system. Feedback travels through every part of it, connecting judgment, reasoning, and search into one evolving system. RULER and ART help your agents evaluate and improve their own behavior, while rLLM gives your entire system the ability to learn and reason as a unified, adaptive intelligence.

## Conclusion

The core lesson of this chapter is that your agent's capability comes from learning dynamics and inference strategy, not model size. Rewards connect actions to consequences, trajectories preserve reasoning, and roll-outs turn experience into structure. With KL as the safety rope, GRPO and GSPO transform relative comparisons into stable policy updates, so agents improve their process, not just their answers.

You then learned how to teach agents efficiently. RULER provides consistent, rubric driven judgment without handcrafted metrics, and ART turns judged trajectories into updates that make tool use more reliable. Hybrid rewards combine hard programmatic checks with qualitative assessments of reasoning, producing a training loop that is interpretable, data efficient, and aligned with how your agents actually operate.

Reasoning does not end at training time. Test time compute changes the game: by allocating more thinking during inference, smaller models can improve by exploring and refining alternative solutions. MCTS organizes that thinking, and AB-MCTS adapts breadth and depth with Bayesian updates, so agents explore when uncertain and refine when confident. Search becomes a first class part of reasoning, not an afterthought.

Finally, you moved from single agents to system level learning. rLLM generalizes the pattern by treating workflows themselves as trainable programs. Planner and executor, solver and judge, and tree search can all produce trajectories, receive rewards, and improve together as a coordinated system. This closes the gap between design and deployment: the

same control flow that runs your agents in production becomes the loop that makes them better.

Taken together, these components demystify the magic behind agency: relative evaluation for signal, KL for stability, ART for trajectory centric training, AB-MCTS for adaptive inference, and rLLM to make the whole workflow learn. In the next chapter, you will learn how to select an LLM and understand the tradeoffs that shape your agents' performance.

- <sup>1</sup> This quote is one of the three laws proposed by the British science fiction writer Arthur C. Clarke.
- <sup>2</sup> Zhihong Shao et al. [“DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.”](#), (2024).
- <sup>3</sup> Chujie Zheng et al. [“Group Sequence Policy Optimization.”](#), (2025).
- <sup>4</sup> Yuichi Inoue et al. (2025). Wider or Deeper? Scaling LLM Inference-Time Compute with Adaptive Branching Tree Search. <https://arxiv.org/abs/2503.04412>